Machine Vision – Final Project Report

Integrated Vision-Guided Robotic Pick-and-Place System

Kossi Aziadzo

Medis Pramoda

Rashmika Dineth Ruwanpathirana

Project link: https://github.com/Rashmika-Dineth/Project_V2/tree/main

1. Introduction

This project presents the design and implementation of a complete vision-guided robotic pick-and-place system, integrating camera-based object detection with a Dobot MG400 industrial robot. The objective was to combine previously developed course components image preprocessing, segmentation, coordinate mapping, and object selection into a robust and usable end to end system.

The final system enables a camera to detect objects placed on a table, compute their real-world coordinates using homography based calibration, and command the robot to pick and place selected objects into a designated box. The system provides both a command-line interface (CLI) and a graphical user interface (GUI) designed for factory operators.

2. System Architecture and Integration

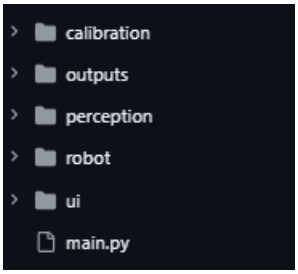
The system was implemented using modular architecture to ensure maintainability and independent testing of each subsystem. The main modules are:

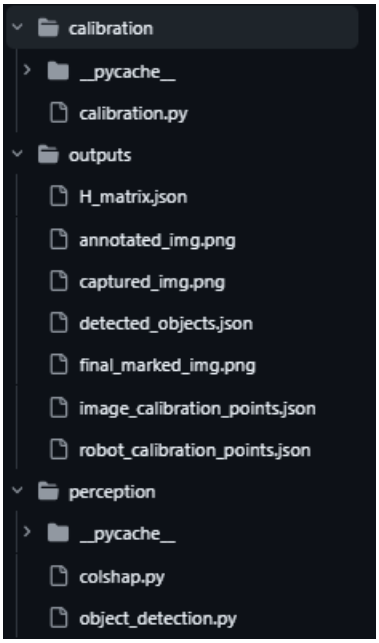
- Calibration module
- Perception (vision) module
- Robot control module
- CLI interface
- Streamlit-based GUI
- Output

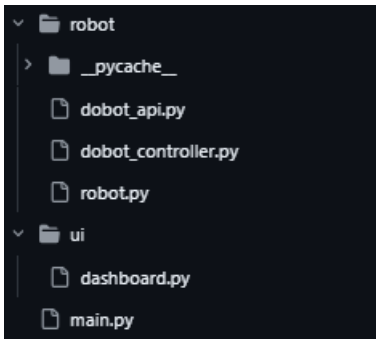
The overall processing pipeline is:

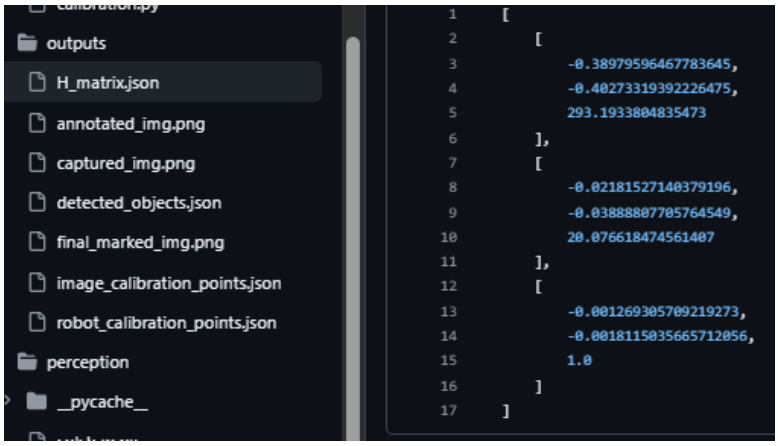
1. Camera image acquisition
2. Image preprocessing and segmentation
3. Object detection and center extraction (u, v)
4. Homography-based transformation to robot coordinates (X, Y)
5. Plan or Execute action

This modular separation allowed each component to be validated independently before full system integration.

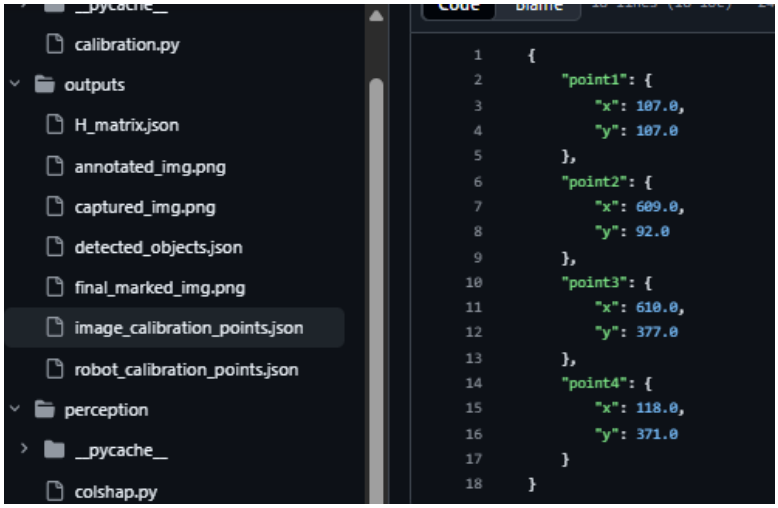




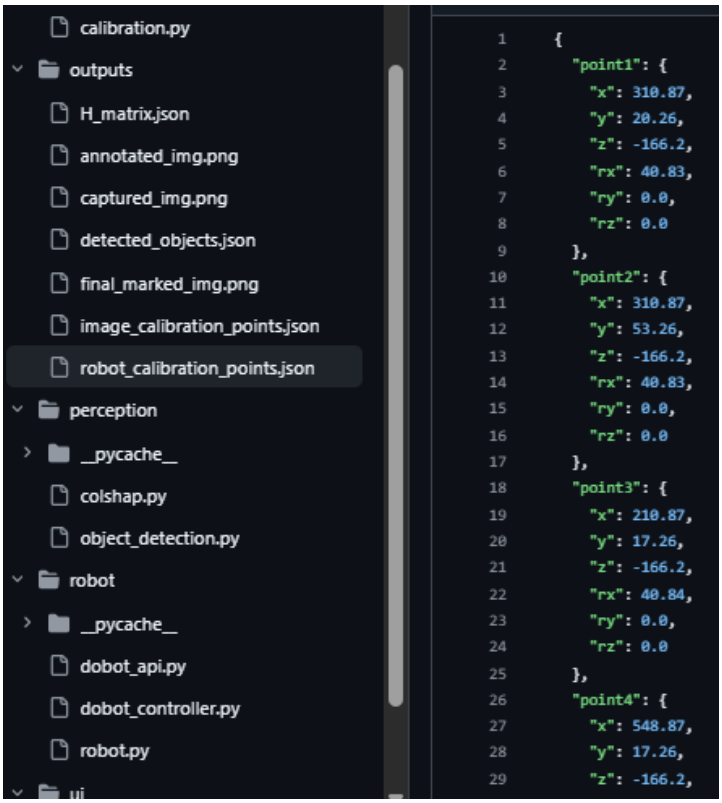





```
[
    [
        -0.38979596467783645,
        -0.40273319392226475,
        293.1933804835473
    ],
    [
        -0.02181527140379196,
        -0.03888807705764549,
        20.076618474561407
    ],
    [
        -0.001269305709219273,
        -0.0018115035665712056,
        1.0
    ]
]
```


```
{
    "point1": {
        "x": 107.0,
        "y": 107.0
    },
    "point2": {
        "x": 609.0,
        "y": 92.0
    },
    "point3": {
        "x": 610.0,
        "y": 377.0
    },
    "point4": {
        "x": 118.0,
        "y": 371.0
    }
}
```


```
{
  "point1": {
    "x": 310.87,
    "y": 20.26,
    "z": -166.2,
    "rx": 40.83,
    "ry": 0.0,
    "rz": 0.0
  },
  "point2": {
    "x": 310.87,
    "y": 53.26,
    "z": -166.2,
    "rx": 40.83,
    "ry": 0.0,
    "rz": 0.0
  },
  "point3": {
    "x": 210.87,
    "y": 17.26,
    "z": -166.2,
    "rx": 40.84,
    "ry": 0.0,
    "rz": 0.0
  },
  "point4": {
    "x": 548.87,
    "y": 17.26,
    "z": -166.2,
```

3. Camera-to-Robot Calibration

A dedicated calibration application was developed to compute the homography matrix between the image plane and the robot workplane.

3.1 Calibration Procedure

The calibration process includes:

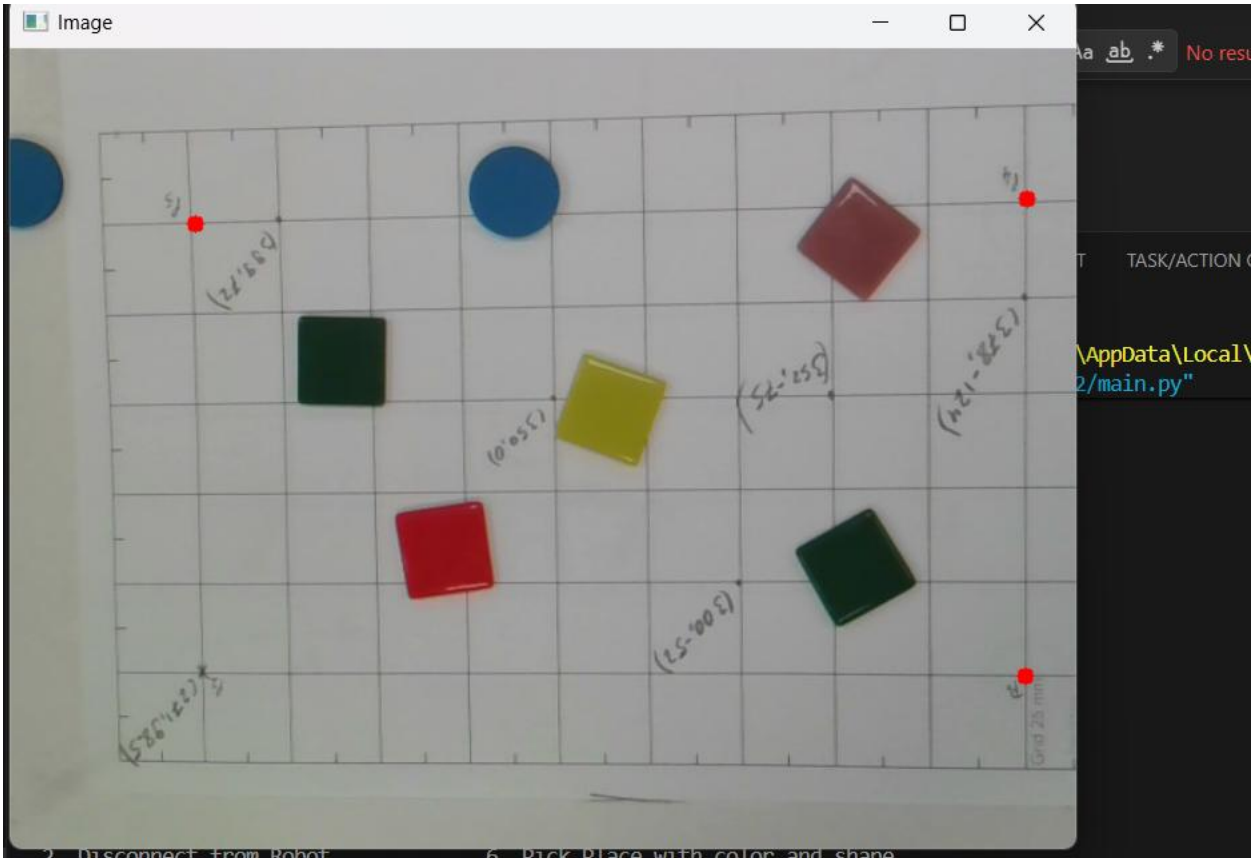
- Capturing an image from the real camera
- Selecting four-pixel coordinates (u, v) and save them in a json file
- Recording corresponding robot coordinates (X, Y) on the table plane and save them in a json file
- Computing a 3×3 homography matrix H
- Saving the calibration results into H_matrix.json

The JSON files for the calibration contains:

- Homography matrix H
- Image calibration points
- Robot calibration points

The calibration file is loaded automatically during detection and picking. This design avoids repeated calibration and improves usability.

The accuracy of object placement strongly depends on calibration precision. Small errors in point selection directly affect the mapping quality.




```
2. Disconnect from Robot                6. Pick Place with color and shape
3. Calibration                          7. Run Streamlit UI
4. Object Detection                     8. Exit
================================================================================
Please select an option (1-8): 3
Starting calibration...
Press SPACE to capture image
Press ESC to exit
Point 1: (111, 105)
Point 2: (610, 90)
Point 3: (609, 376)
```

Capturing image from the robot actual coordinates

```python
##########################################################################################
# Read and Save Calibration Points (CMD)
##########################################################################################
def Get_Robot_Calibration_Points():
    try:
        calibration_points = {}

        for i in range(6):
            if i == 4:
                input("\nMove robot to on top of a tile and press Enter...")
            elif i == 5:
                input("\nMove robot to on top of the drop location and press Enter...")
            else:
                input(f"\nMove robot to point {i+1} and press Enter...")

            current_pos = GetCurrentPosition()

            x  = float(current_pos[0])
```

```
Please select an option (1-8): 3
Starting calibration...
Press SPACE to capture image
Press ESC to exit
Point 1: (158, 105)
Point 2: (611, 93)
Point 3: (611, 375)
Point 4: (167, 377)
Image points saved to ./outputs/image_calibration_points.json

Move robot to point 1 and press Enter...
```

This need to replace with image

```
Generating homography matrix...
Homography matrix generated and saved.
[[-3.93095974e-01 -3.97282068e-01  2.93809130e+02]
 [-2.21031216e-02 -3.81970814e-02  2.00499384e+01]
 [-1.27973059e-03 -1.78454700e-03  1.00000000e+00]]
Calibration completed successfully.
```

Capture the real drop off location and get the real pick z location

```
##########################################################################################
# Load the Json file
##########################################################################################
def Load_DROP_Data():
    global DROP_POINT, DROP_POINT_UP
    print("\nUpdating Dropoff location robot calibration data...")
    with open(robot_calibration_point_file, "r") as f:
        data = json.load(f)
        DROP_POINT = [data["point6"]["x"], data["point6"]["y"], data["point6"]["z"], data["point6"]["r
        DROP_POINT_UP = [data["point6"]["x"], data["point6"]["y"], data["point6"]["z"] + SAFE_Z_OFFSET
        #  DROP_POINT[0] = data["point6"]["z"]
```

```
SAFE_Z_OFFSET = 60        # distance above object
PICK_Z = 0                # object surface height

##########################################################################################
# Load the Z data
##########################################################################################
def Load_Z_Data():
    global PICK_Z
    print("\nLoading robot calibration data...")
    with open(robot_calibration_point_file, "r") as f:
        data = json.load(f)
        PICK_Z = data["point5"]["z"]
        return PICK_Z

##########################################################################################
```

Keep the path variables separate

```
##########################################################################################
# File paths
##########################################################################################
IMAGE_PATH = "./outputs/captured_img.png"
H_MATRIX_PATH = "./outputs/H_matrix.json"
OUTPUT_JSON_PATH = "./outputs/detected_objects.json"
OUTPUT_ANNOTATED_IMAGE = "./outputs/annotated_img.png"
OUTPUT_ANNOTATED_IMAGE_CENTER="./outputs/mapping_visualized.png"
robot_calibration_point_file = "./outputs/robot_calibration_points.json"

SAFE_Z_OFFSET = 60        # distance above object
PICK_Z = 0                # object surface height
```

4. Vision-to-Robot Mapping Pipeline

The perception module detects objects using classical image processing techniques.

4.1 Image Processing Steps

1. Image acquisition
2. Gaussian blurring
3. Thresholding or color-based segmentation
4. Object shape detection
5. Object center computation

For each detected object, the pixel center (u, v) is calculated. The homography matrix H is then applied:

$$(X, Y, 1)^T = H \cdot (u, v, 1)^T$$

The resulting (X, Y) coordinates are used as robot targets.

4.2 Optional Object Selection

The system supports optional filtering by:

- **Color** (e.g., red, blue, yellow objects)
- **Shape** (e.g., circle, square, rectangle)
- Combined logic (e.g., blue circles)

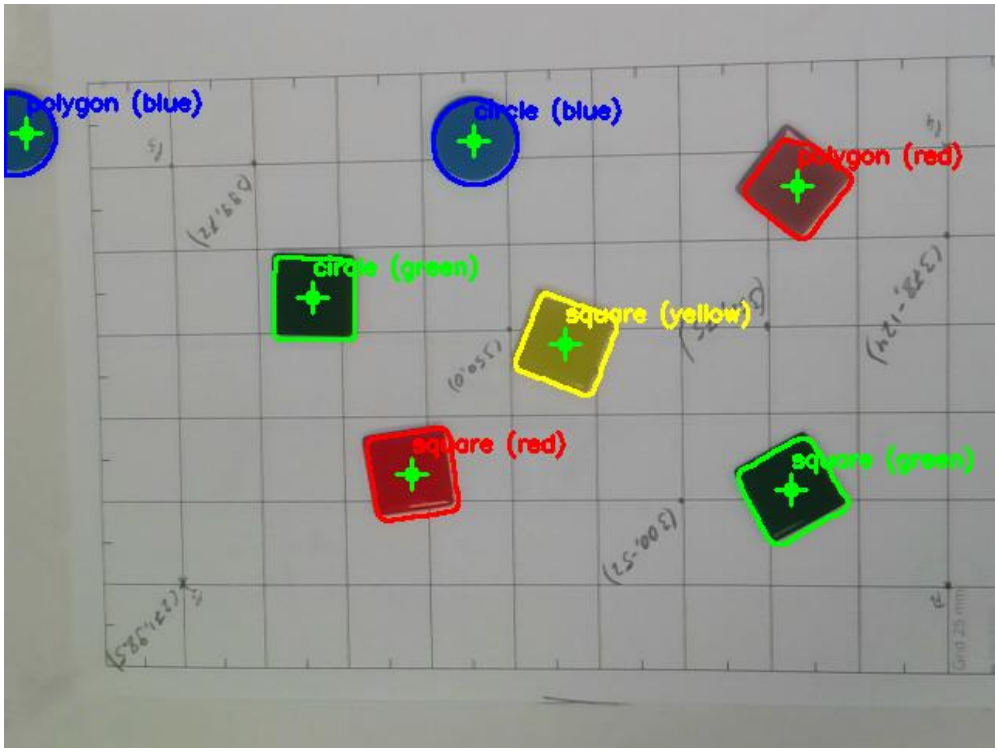
This increases flexibility and simulates industrial sorting tasks.

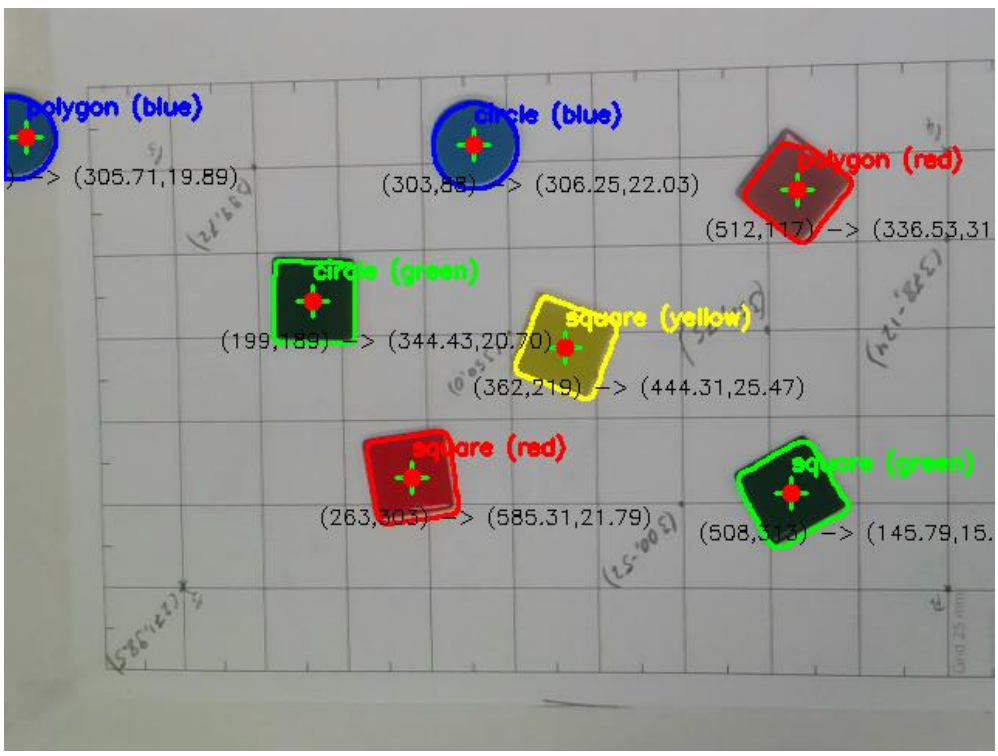
4.3 Output Visualization

The system generates an annotated overlay image that includes:

- Object center marker
- Pixel coordinates
- Converted robot coordinates

In CLI mode, this image is saved to disk.
In GUI mode, it is displayed directly to the operator.


polygon (blue)
circle (blue)
polygon (red)
circle (green)
square (yellow)
square (red)
square (green)


polygon (blue)
) -> (305.71,19.89)
circle (blue)
(303,88) -> (306.25,22.03)
polygon (red)
(512,117) -> (336.53,31
circle (green)
(199,189) -> (344.43,20.70)
square (yellow)
(362,219) -> (444.31,25.47)
square (red)
(263,303) -> (585.31,21.79)
square (green)
(508,313) -> (145.79,15.

5. Plan Mode vs Execute Mode

A key design requirement was safe separation between simulation and real robot motion.

Plan Mode (Without Dobot Connection)

- Detect objects
- Compute and display (X, Y) coordinates
- Generate overlay image
- Do not move the robot

Plan mode allows validation of detection and coordinate mapping without risk.

Execute Mode (Execution After Dobot connection)

- Perform full pick-and-place sequence
- Move robot to target
- Activate gripper
- Move to drop location
- Release object

Execute mode is gated:

- CLI requires explicit --mode execute
- GUI requires Execute toggle and confirmation

This design ensures operational safety.

6. Command-Line Interface (CLI)

The CLI provides full system functionality and was implemented as the mandatory operator interface.

Example usage:

python main.py

The CLI:

- Loads calibration automatically
- Prints pixel and robot coordinates
- Displays number of targets found
- Saves annotated overlay images
- Provides clear status messages

The interface was designed to be simple, readable, and safe.

```
================================================================================
           Machine Vision Project - Dobot MG400
================================================================================
1. Connect to Robot                  5. Object Pick and Place
2. Disconnect from Robot             6. Pick Place with color and shape
3. Calibration                       7. Run Streamlit UI
4. Object Detection                  8. Exit
================================================================================
Please select an option (1-8):
```

```
================================================================================
1. Connect to Robot                 5. Object Pick and Place
2. Disconnect from Robot            6. Pick Place with color and shape
3. Calibration                      7. Run Streamlit UI
4. Object Detection                 8. Exit
================================================================================
Please select an option (1-8): 6
Starting pick and place operation based on color and shape...
Select object shape to pick and place:
1. Circle
2. Square
3. Triangle
4. All shapes
Enter shape choice (1-4): 2

Select object color to pick and place:
1. Red
2. Green
3. Blue
4. Yellow
5. All colors
Enter color choice (1-5): 2

Running pick and place for shape: square, color: green...

Found 1 objects to pick
Moving to point: [350, 0, 0, 0]
```

7. Graphical User Interface (GUI)

A Streamlit-based GUI was developed to simulate a factory operator panel.

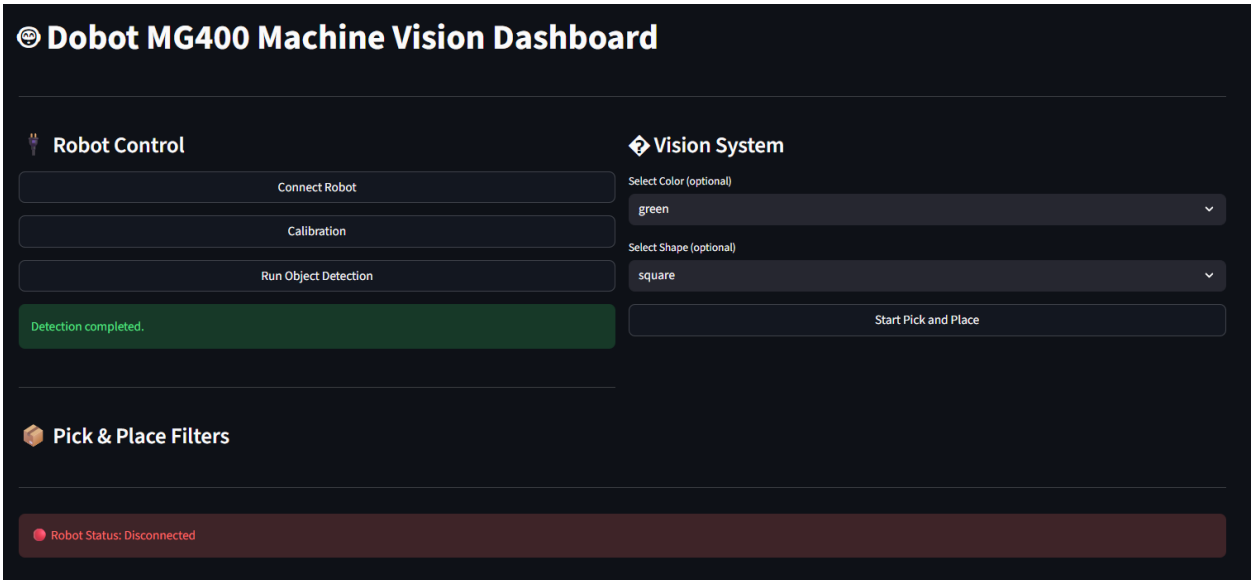
The GUI provides:

- Live or refreshed camera image
- Mode selection (Plan / Execute)
- Optional color and shape dropdown selection
- Buttons:
    - Capture / Refresh
    - Detect Target
    - Run Pick (enabled only in Execute mode)

The interface displays:

- Annotated image overlay
- Computed robot coordinates
- Clear feedback messages

The GUI improves usability compared to the CLI and better represents real industrial systems.


Dobot MG400 Machine Vision Dashboard
Robot Control
Connect Robot
Calibration
Run Object Detection
Detection completed.
Vision System
Select Color (optional)
green
Select Shape (optional)
square
Start Pick and Place
Pick & Place Filters
Robot Status: Disconnected

Vision System
Captured Image
Annotated Image
Marked Annotated Image

8. System Performance and Evaluation

The system successfully integrates calibration, perception, coordinating transformation, and robot control.

Detection performance:

- Reliable under moderate lighting conditions
- Sensitive to strong shadows and reflections

Mapping accuracy:

- Dependent on calibration quality
- Small millimeter-level errors observed due to manual point selection

Execution performance:

- Smooth robot movement
- Clear separation between simulation and physical motion

Overall, the system performs reliably for controlled laboratory conditions.

9. Discussion

The integration phase required careful debugging of coordinate frames and communication between modules. The most challenging aspects were:

- **Connecting and communicating with DOBOT using the Python API:**
  There were some issues when trying to connect to DOBOT using the provided API. Although the ping command was successful and the DOBOT communicated properly with the DOBOT application, some DOBOT devices rejected the API connection itself. The application was also developed to maintain the connection with the DOBOT during program execution and to disconnect once the program is completed or terminated by the user.

- **Implementing safe execution with separate modules:**
  Some modules had to be kept separate, and due to the folder structure, there were missing path errors that took additional time to troubleshoot.

- **Color and shape detection:**
  The results were affected by lighting conditions, filtering methods, the type of module, and the mask used. It was also identified that different camera types produce different outputs. In some cases, squares were detected as polygons; however, this error was minimized by optimizing the shape detection code.

- **Camera quality:**
  During the image capture process, the Streamlit camera was initially planned for use with the web interface. However, the OpenCV camera provided better filtering and overall performance compared to the Streamlit camera. Therefore, the code was developed using OpenCV for image capture.

The project demonstrated how individual computer vision techniques become significantly more complex when integrated into a real robotic system.

From a practical perspective, the system is functional but could be improved by:

- Automatic chessboard-based calibration
- Improved illumination control
- Multi-object trajectory optimization
- Error recovery mechanisms

The development process provided valuable insight into real-world industrial automation challenges.

10. Conclusion

This project successfully implemented a complete vision-guided robotic pick-and-place system using camera calibration, object detection, coordinate mapping, and robotic control.

Both CLI and GUI interfaces operate correctly, and the Plan/Execute mode separation ensures safety and usability. The modular architecture supports further expansion and industrial adaptation.

The project demonstrates the practical application of machine vision techniques in robotic automation systems.